

# AgentCore GitLab Tool: Loop Behaviour, Jira Side Effects & GitOps Scope Expansion

---

Prepared for: Backend Engineer

Date: May 15, 2027

## System Overview

---

The AgentCore agent is a Bedrock-hosted agent exposed to the team via Slack. When a message is sent in the relevant Slack channel, an SQS-backed Lambda called `agentcore-invoker` receives the event, strips the bot mention from the message, derives a `session_id` by hashing `channel:thread_ts`, and forwards the prompt to AgentCore via `invoke_agent_runtime`. AgentCore runs the agent turn, invoking whatever tools it needs, and returns a single `output` string when it is done. The invoker then posts that string back to the Slack thread. The invoker has no visibility into what happens inside the AgentCore session — it fires the prompt and waits. There is no streaming, no intermediate state, and no mechanism for the invoker to detect or interrupt anything mid-execution.

The agent has two tools, each backed by its own Lambda:

- `jira_tool` — handles everything on the project management side: creating tickets, fetching them, updating status/priority/assignee, listing tickets, adding comments, and listing users.
- `gitlab_tool` — handles the code side: resolving a GitLab project by name, browsing the repo tree, reading files, creating branches, committing files, and opening Draft MRs.

Credentials for both are fetched from AWS Secrets Manager and cached in memory for five minutes.

## Intended End-to-End Flow

---

The following is the correct, expected sequence the agent should execute for every infrastructure or GitOps change request. This is the target behaviour all fixes and additions should be validated against.

1. Create a Jira ticket in the **DEVOPS** project with status **In Progress**, capturing the nature of the request in the summary and description.
2. Resolve the target GitLab project by name, routing to the correct subgroup (`terraform` or GitOps) depending on the type of request.
3. Browse the repository tree and read an existing reference file to understand the conventions before generating anything.
4. Create a branch named after the Jira ticket key (e.g. `feat/DEVOPS-42`).
5. Commit the generated file (Terragrunt HCL or Kubernetes/Flux manifest) to that branch.

**NOTE: IF YOU MODIFY THE INSTRUCTIONS VARIABLES INPUT, IT SHOULDN'T BE MORE THAN 2000 CHARACTERS.**

6. Open a Draft MR targeting `main`. The MR description must include a human-readable summary of the change and a hyperlink back to the Jira ticket (e.g. `Jira: DEVOPS-42` linking to the Atlassian URL).
7. Post a comment on the Jira ticket containing the branch name, commit SHA, and a direct link to the GitLab MR.
8. Return a final response to the user in Slack with the full details: the Jira ticket key and link, the MR title and link, the branch name, and a short summary of what was created. The user should be able to navigate to both Jira and GitLab directly from the Slack message without searching.

---

*Steps 6, 7, and 8 are currently not executing reliably due to the loop described in Issue 1 below. The agent stalls during the repository exploration phase and either never reaches MR creation, or restarts the entire sequence mid-session.*

---

## Issue 1 — GitLab Tool Enters a Tree-Reading Loop

---

### What is happening

Looking at the GitLab Lambda logs from `2026-05-11T23:47:28` onward, the agent repeatedly calls `read_gitlab_tree` and `read_gitlab_file` on the same paths within a single logical task. The sequence:

```
resolve → tree("") → tree("prod") → tree("prod/s3") →  
file("prod/s3/remote-work-assets/terragrunt.hcl")
```

is executed at least twice before the agent eventually moves forward. Individual paths like `prod/s3` and the root tree are re-read multiple times across separate Lambda invocations within the same session, and the agent never cleanly exits the exploration phase to proceed with branch creation.

### The tool field is never present

Every single Lambda invocation logs `tool_inferred_from_payload` as a WARNING. This means the `tool` field that AgentCore Gateway is supposed to inject into the event is never actually present in production. The tool is being identified on every call purely by inspecting the shape of the payload keys via the `_infer_tool()` fallback function. This fallback was written defensively but was never intended to be the primary dispatch path.

This is significant because `read_gitlab_tree` and `read_gitlab_file` have overlapping key signatures. Both receive `project_id`, `ref`, and either a `path` or `file_path` key. The inference logic distinguishes them by checking whether a `content` key is also present. If the agent receives a response it does not recognise as terminal, it will re-enter the exploration phase instead of advancing.

**NOTE: IF YOU MODIFY THE INSTRUCTIONS VARIABLES INPUT, IT SHOULDN'T BE MORE THAN 2000 CHARACTERS.**

## What to investigate

- Verify the AgentCore Gateway target configuration — specifically whether `inline_payload` is set correctly and whether the `tool` field injection is enabled. The `tool` field being absent on every invocation suggests either a misconfiguration or that the Lambda is receiving events through a path that bypasses Gateway injection.
- Determine what the agent is receiving back from the tool responses and whether anything in the response format or content causes the agent to treat the exploration phase as incomplete.
- Review the agent instructions in AgentCore to understand whether there is a step limit or termination condition on the exploration phase, or whether the agent is free to keep calling tree/file tools indefinitely.

## Issue 2 — Loop Causes Duplicate Jira Ticket Creation

---

### What is happening

Because `session_id` is derived from `channel:thread_ts`, all messages within the same Slack thread share an AgentCore session. The agent carries memory within that session. When the GitLab tool loops and the agent loses track of where it is in the workflow, it does not restart completely blind — it replays within the same session context. This is why the Jira logs show the agent re-invoking `create_jira_ticket` with a slightly rephrased summary rather than an identical one. It remembers it already attempted the action but does not recognise the prior attempt as a completed success, so it tries again with different wording.

### What the logs show

- At `23:47:28` — agent creates ticket: "Create S3 bucket vrzn-use1-platform in prod for verizon-users" — **ticket\_created**. Legitimate first creation.
- At `23:48:28` — agent retries mid-loop with "Add S3 bucket vrzn-use1-platform for verizon-users in prod" — **ticket\_created**. Idempotency check misses because the summary wording is different. A second real ticket is created.
- At `23:49:29` — agent tries a third time with the original summary — **ticket\_already\_exists**. Idempotency check correctly fires this time.

The result is two real Jira tickets for the same piece of work. This is a symptom of the GitLab loop, not an independent Jira bug. Fixing the loop should stop the agent re-entering the ticket creation step.

### Secondary concern — idempotency check reliability

The idempotency check in `_find_existing_ticket()` performs a case-insensitive exact match on the summary string. Minor rephrasing by the model is sufficient to bypass it, as seen above. This is worth noting independently of the loop fix.

**NOTE: IF YOU MODIFY THE INSTRUCTIONS VARIABLES INPUT, IT SHOULDN'T BE MORE THAN 2000 CHARACTERS.**

## Issue 3 — GitOps Repos Are Unreachable

---

### What is happening

The `GITLAB_GROUP_PATH` environment variable is set to `audasity-inc/devops`. Inside `_resolve_gitlab_project`, the direct-lookup path is constructed as `{GITLAB_GROUP_PATH}/terraform/{project_name}`. This is visible in every resolve call in the logs:

```
RESOLVE_ATTEMPT_DIRECT: path=audasity-inc/devops/terraform/remote-work
```

The `/terraform/` segment is hardcoded unconditionally, meaning every project resolution attempt, regardless of request type, targets the Terraform subgroup. The agent currently has no way to reach a GitOps repository.

### Repo structure

Both Terraform and GitOps repos live under the same parent group. Their paths are:

- Terraform/Terragrunt: `audasity-inc/devops/terraform/<project-name>`
- GitOps (Flux/Kubernetes manifests):  
`audasity-inc/devops/<gitops-subgroup>/<project-name>` — exact subgroup name to be confirmed with the team.

### Scope of changes needed

- The tool needs to understand the type of request it is handling — infrastructure (Terraform/Terragrunt) versus GitOps (Flux/Kubernetes manifests) — and route the project resolution to the correct subgroup path.
- The content guard in `commit_gitlab_file` that blocks raw Terraform resource block markers (`resource "aws_"`, `terraform { backend`, etc.) is appropriate for Terraform repos but must not fire when the agent is writing Kubernetes or Flux manifests to a GitOps repo. The guard needs to be aware of which repo type it is operating against.
- Any change to how subgroup routing is handled will also require an update to `variables.tf` and the environment variable wiring in `main.tf`.

## Summary

---

The three issues are related but separable and can be addressed independently.

- **Issue 1 (loop)** is the root cause of Issue 2 and should be addressed first. The `tool` field injection should be verified at the AgentCore Gateway configuration level before assuming the fix lives inside `gitlab_tool.py`.
- **Issue 2 (Jira duplicates)** should resolve once the loop is fixed. The idempotency check's exact-match limitation is a secondary concern worth tracking separately.
- **Issue 3 (GitOps scope)** is a net-new capability and is independent of the loop. It requires changes to project resolution logic, content validation, and Terraform variable configuration.

**NOTE: IF YOU MODIFY THE INSTRUCTIONS VARIABLES INPUT, IT SHOULDN'T BE MORE THAN 2000 CHARACTERS.**